Sum of Sortable Integers

Problem

You are given an integer array **nums** of length **n**. An integer **k** is called **sortable** if **k** divides **n** and you can sort **nums** in non-decreasing order by sequentially performing the following operations:

1. Partition **nums** into consecutive subarrays of length **k**.
2. Cyclically rotate each subarray independently any number of times to the left or to the right.

Return an integer denoting the sum of all possible sortable integers **k**.

Examples

Example 1:
Input: nums = [3,1,2]
Output: 3
Explanation: For k = 3, the block [3,1,2] can be rotated once to [1,2,3]. Only k = 3 is sortable.

Example 2:
Input: nums = [7,6,5]
Output: 0
Explanation: Neither k = 1 nor k = 3 can make the array non-decreasing.

Example 3:
Input: nums = [5,8]
Output: 3
Explanation: The array is already sorted, so both divisors 1 and 2 are sortable.

Approach

A divisor k is sortable only if every length-k block can be cyclically rotated into non-decreasing order, and every later block's minimum value is at least the maximum value of all earlier blocks. A circular sequence can be rotated into non-decreasing order exactly when it has at most one circular descent.

Precompute the prefix maximum, suffix minimum, and prefix count of adjacent descents. Then check each divisor k by examining its blocks in constant time.

Java Solution

```
class Solution {
    public int sortableIntegers(int[] nums) {
        int n = nums.length;

        int[] prefixMax = new int[n + 1];
        int[] suffixMin = new int[n + 1];
        int[] prefixDrops = new int[n + 1];

        prefixMax[0] = 0;
        suffixMin[n] = Integer.MAX_VALUE;

        for (int i = 0; i < n; i++) {
            prefixMax[i + 1] = Math.max(prefixMax[i], nums[i]);
        }

        for (int i = n - 1; i >= 0; i--) {
            suffixMin[i] = Math.min(suffixMin[i + 1], nums[i]);
        }
        for (int i = 1; i < n; i++) {
```

```
            prefixDrops[i] = prefixDrops[i - 1]
                    + (nums[i - 1] > nums[i] ? 1 : 0);
        }

        long answer = 0;

        for (int k = 1; k <= n; k++) {
            if (n % k != 0) {
                continue;
            }

            if (isSortable(nums, k, prefixMax, suffixMin, prefixDrops)) {
                answer += k;
            }
        }

        return (int) answer;
    }

    private boolean isSortable(
            int[] nums,
            int k,
            int[] prefixMax,
            int[] suffixMin,
            int[] prefixDrops
    ) {
        int n = nums.length;

        for (int start = 0; start < n; start += k) {
            int end = start + k - 1;

            int circularDrops =
                    prefixDrops[end] - prefixDrops[start];

            if (nums[end] > nums[start]) {
                circularDrops++;
            }

            if (circularDrops > 1) {
                return false;
            }

            if (start > 0
                    && prefixMax[start] > suffixMin[start]) {
                return false;
            }
        }

        return true;
    }
}
```

Complexity

Time complexity: O(n + sigma(n)), where sigma(n) is the sum of divisors of n. For each divisor k, the solution checks n / k blocks.
Space complexity: O(n) for the prefix maximum, suffix minimum, and prefix descent arrays.